Logistic Regression and Random Forest to Determine Credit Card Default

Travis Gubbe

December 29, 2025

In this session, I examine the Credit Card Clients data set found on the UCI Machine Learning Repository website to determine if a person will default on their credit card using logistic regression and random forest.

About the Data Set

The data set can be found on the UCI Machine Learning Repository site at the following link: https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients.

The data contains 24 variables and a total of 30,000 individual instances. The variables are:

- Default Payment (0 = No, 1 = Yes)
- Amount of Given Credit
- Gender (1 = Male, 2 = Female)
- Education (1 = Graduate School, 2 = University, 3 = High School, 4 = Other, 5 = Unknown)
- Marital Status (1 = Married, 2 = Single, 3 = Others)
- Age
- History of Past Payment from April 2005 to September 2005 where -2=no consumption, -1=pay duly, 0=the use of revolving credit, 1=payment delay for one month, 2=payment delay for two months, ... 9=payment delay for nine months and above
- Amount of Bill Statement from April 2005 to September 2005
- Amount of Previous Payment April 2005 to September 2005

There is a separate variable for each past payment, bill statement, and previous payment from April to September.

Exploratory Analysis

First, the data set is loaded into a data frame named "credit". The data frame is also viewed to see the columns and class types.

```
data <- read.csv("~/R datasets/Credit_Card.csv")
credit = subset(data, select = c("Limit_Bal", "Sex", "Education", "Marriage", "Age", "Pay_Sep",
                                 "Pay_Aug", "Pay_July", "Pay_June", "Pay_May", "Pay_April",
                                 "Bill_Amt_Sep", "Bill_Amt_Aug", "Bill_Amt_July",
                                 "Bill_Amt_June", "Bill_Amt_May", "Bill_Amt_April",
                                 "Pay_Amt_Sep", "Pay_Amt_Aug", "Pay_Amt_July","Pay_Amt_June",
                                 "Pay_Amt_May", "Pay_Amt_April", "Default"))
#Inspect the data frame
head(credit)
```

```
##   Limit_Bal Sex Education Marriage Age Pay_Sep Pay_Aug Pay_July Pay_June
## 1     20000   2         2        1  24       2       2       -1       -1
## 2    120000   2         2        2  26      -1       2        0        0
```

```
## 3     90000   2         2        2  34       0       0        0        0
## 4     50000   2         2        1  37       0       0        0        0
## 5     50000   1         2        1  57      -1       0       -1        0
## 6     50000   1         1        2  37       0       0        0        0
##   Pay_May Pay_April Bill_Amt_Sep Bill_Amt_Aug Bill_Amt_July Bill_Amt_June
## 1      -2        -2         3913         3102           689             0
## 2       0         2         2682         1725          2682          3272
## 3       0         0        29239        14027         13559         14331
## 4       0         0        46990        48233         49291         28314
## 5       0         0         8617         5670         35835         20940
## 6       0         0        64400        57069         57608         19394
##   Bill_Amt_May Bill_Amt_April Pay_Amt_Sep Pay_Amt_Aug Pay_Amt_July Pay_Amt_June
## 1            0              0           0         689            0            0
## 2         3455           3261           0        1000         1000         1000
## 3        14948          15549        1518        1500         1000         1000
## 4        28959          29547        2000        2019         1200         1100
## 5        19146          19131        2000       36681        10000         9000
## 6        19619          20024        2500        1815          657         1000
##   Pay_Amt_May Pay_Amt_April Default
## 1           0             0       1
## 2           0          2000       1
## 3        1000          5000       0
## 4        1069          1000       0
## 5         689           679       0
## 6        1000           800       0
```

```
#Inspect the classes of the data frame
sapply(credit, class)
```

```
##      Limit_Bal            Sex      Education       Marriage            Age
##      "integer"      "integer"      "integer"      "integer"      "integer"
##        Pay_Sep        Pay_Aug       Pay_July       Pay_June        Pay_May
##      "integer"      "integer"      "integer"      "integer"      "integer"
##      Pay_April   Bill_Amt_Sep   Bill_Amt_Aug  Bill_Amt_July  Bill_Amt_June
##      "integer"      "integer"      "integer"      "integer"      "integer"
##   Bill_Amt_May Bill_Amt_April    Pay_Amt_Sep    Pay_Amt_Aug   Pay_Amt_July
##      "integer"      "integer"      "integer"      "integer"      "integer"
##   Pay_Amt_June    Pay_Amt_May  Pay_Amt_April        Default
##      "integer"      "integer"      "integer"      "integer"
```

```
attach(credit)
```

Next, I want to see the amount of missing values and duplicates in the data frame.

```
sum(is.na(credit))
```

```
## [1] 0
```

```
duplicates <- credit%>%duplicated()
duplicates_amount <- duplicates%>%(table)
duplicates_amount
```

```
## .
## FALSE  TRUE
## 29965    35
```

Since there are 35 duplicates in the data, the data frame is filtered to remove the duplicates.

```
credit <- credit%>%distinct()
#Displays how many duplicates are present in the updated data frame.
duplicates_counts_unique <- credit%>%duplicated()%>%table()
duplicates_counts_unique
```

```
## .
## FALSE
## 29965
```

Next, the factor variables are converted from their numeric values to their actual names. This is done on a copy of the credit data frame.

```
credit1 <- data.frame(credit)
head(credit1)
```

```
##   Limit_Bal Sex Education Marriage Age Pay_Sep Pay_Aug Pay_July Pay_June
## 1     20000   2         2        1  24       2       2       -1       -1
## 2    120000   2         2        2  26      -1       2        0        0
## 3     90000   2         2        2  34       0       0        0        0
## 4     50000   2         2        1  37       0       0        0        0
## 5     50000   1         2        1  57      -1       0       -1        0
## 6     50000   1         1        2  37       0       0        0        0
##   Pay_May Pay_April Bill_Amt_Sep Bill_Amt_Aug Bill_Amt_July Bill_Amt_June
## 1      -2        -2         3913         3102           689             0
## 2       0         2         2682         1725          2682          3272
## 3       0         0        29239        14027         13559         14331
## 4       0         0        46990        48233         49291         28314
## 5       0         0         8617         5670         35835         20940
## 6       0         0        64400        57069         57608         19394
##   Bill_Amt_May Bill_Amt_April Pay_Amt_Sep Pay_Amt_Aug Pay_Amt_July Pay_Amt_June
## 1            0              0           0         689            0            0
## 2         3455           3261           0        1000         1000         1000
## 3        14948          15549        1518        1500         1000         1000
## 4        28959          29547        2000        2019         1200         1100
## 5        19146          19131        2000       36681        10000         9000
## 6        19619          20024        2500        1815          657         1000
##   Pay_Amt_May Pay_Amt_April Default
## 1           0             0       1
## 2           0          2000       1
## 3        1000          5000       0
## 4        1069          1000       0
## 5         689           679       0
## 6        1000           800       0
```

```
#Rename factor variables to their appropriate settings
credit1$Sex[credit$Sex %in% "1"] = "Male"
credit1$Sex[credit$Sex %in% "2"] = "Female"

credit1$Education[credit$Education %in% "1"] = "Grad School"
credit1$Education[credit$Education %in% "2"] = "College"
credit1$Education[credit$Education %in% "3"] = "High School"
credit1$Education[credit$Education %in% "4"] = "Other"
credit1$Education[credit$Education %in% "5"] = "Unknown"

credit1$Marriage[credit$Marriage %in% "0"] = "Unknown"
```

```
credit1$Marriage[credit$Marriage %in% "1"] = "Married"
credit1$Marriage[credit$Marriage %in% "2"] = "Single"
credit1$Marriage[credit$Marriage %in% "3"] = "Other"

credit1$Default[credit$Default %in% "0"] = "No"
credit1$Default[credit$Default %in% "1"] = "Yes"
#See the change in the variable names
head(credit1)
```

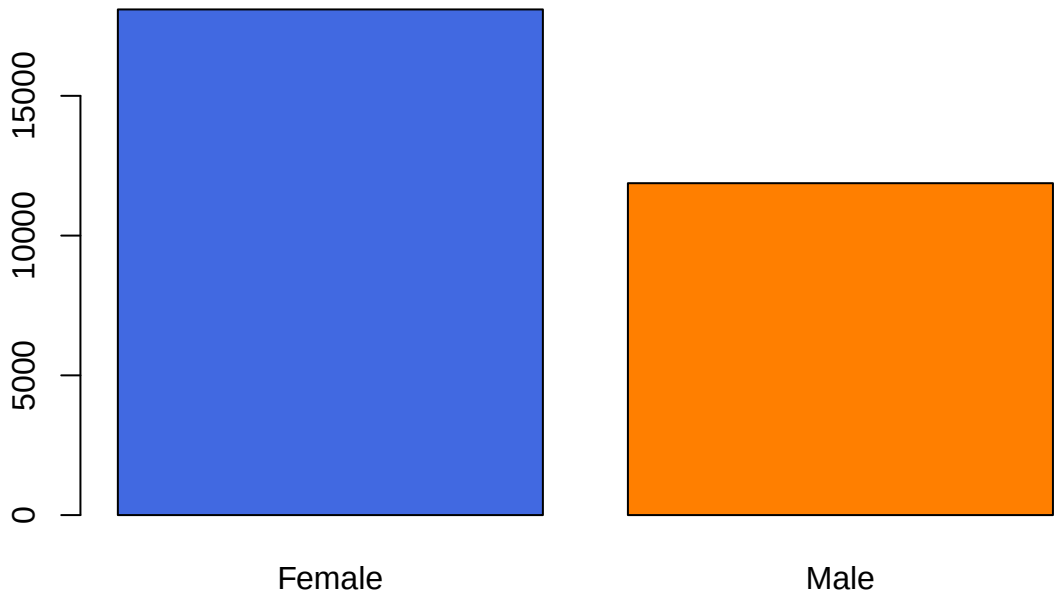
```
##   Limit_Bal    Sex   Education Marriage Age Pay_Sep Pay_Aug Pay_July Pay_June
## 1     20000 Female     College  Married  24       2       2       -1       -1
## 2    120000 Female     College   Single  26      -1       2        0        0
## 3     90000 Female     College   Single  34       0       0        0        0
## 4     50000 Female     College  Married  37       0       0        0        0
## 5     50000   Male     College  Married  57      -1       0       -1        0
## 6     50000   Male Grad School   Single  37       0       0        0        0
##   Pay_May Pay_April Bill_Amt_Sep Bill_Amt_Aug Bill_Amt_July Bill_Amt_June
## 1      -2        -2         3913         3102           689            0
## 2       0         2         2682         1725          2682         3272
## 3       0         0        29239        14027         13559        14331
## 4       0         0        46990        48233         49291        28314
## 5       0         0         8617         5670         35835        20940
## 6       0         0        64400        57069         57608        19394
##   Bill_Amt_May Bill_Amt_April Pay_Amt_Sep Pay_Amt_Aug Pay_Amt_July Pay_Amt_June
## 1            0              0           0         689            0           0
## 2         3455           3261           0        1000         1000        1000
## 3        14948          15549        1518        1500         1000        1000
## 4        28959          29547        2000        2019         1200        1100
## 5        19146          19131        2000       36681        10000        9000
## 6        19619          20024        2500        1815          657        1000
##   Pay_Amt_May Pay_Amt_April Default
## 1           0             0     Yes
## 2           0          2000     Yes
## 3        1000          5000      No
## 4        1069          1000      No
## 5         689           679      No
## 6        1000           800      No
```

Next, exploratory tables are made to view the distribution of the data set.

Data Distribution

Next, bar plots and distribution tables are created to see the proportion of the variables. This is done to see if the data is normally distributed. If the data is not normally distributed, it's advantageous to see how the data is skewed.

```
#View the bar plots for the amount for each categorical variable
counts_Sex <- table(credit1$Sex)
barplot(counts_Sex, col = c("royalblue", "darkorange1"))
```



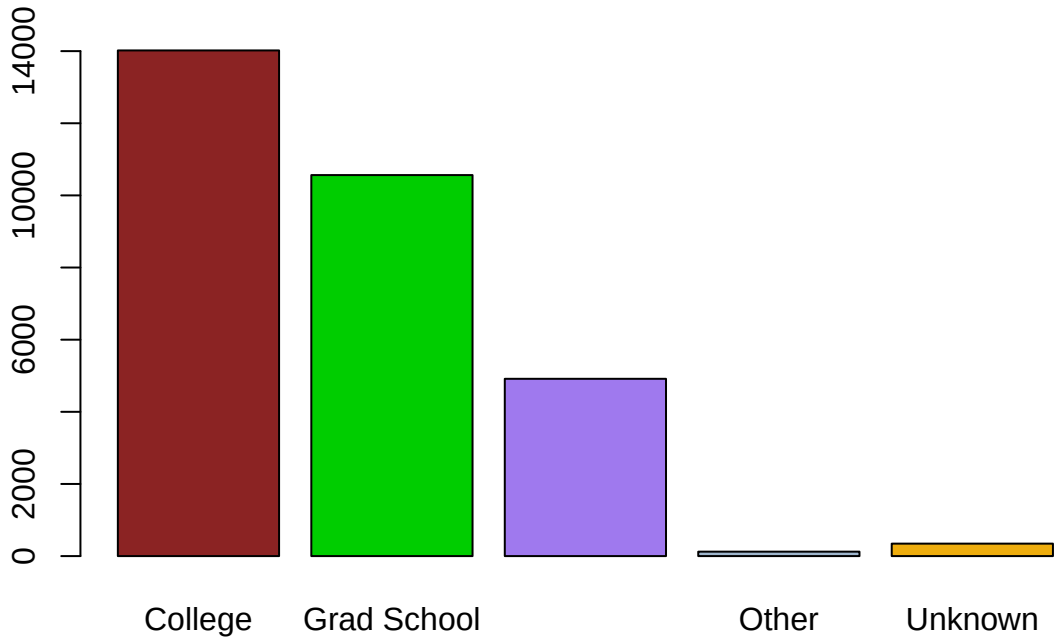

```
#Basic table view of the amount of males and females
table(credit1$Sex)
```

```
##
## Female   Male
##  18091  11874
```

```
#Proportion of each gender in table
prop.table(counts_Sex)
```

```
##
##    Female      Male
## 0.6037377 0.3962623
```

```
counts_Education <- table(credit1$Education)
barplot(counts_Education, col = c("brown4", "green3", "mediumpurple2", "slategray3",
                                  "darkgoldenrod2"))
```




```
table(credit1$Education)
```

```
##
##     College Grad School High School       Other     Unknown
##       14019       10563        4915         123         345
```

```
#Proportion of each education level in table
prop.table(counts_Education)
```

```
##
##     College Grad School High School       Other     Unknown
## 0.467845820 0.352511263 0.164024695 0.004104789 0.011513432
```

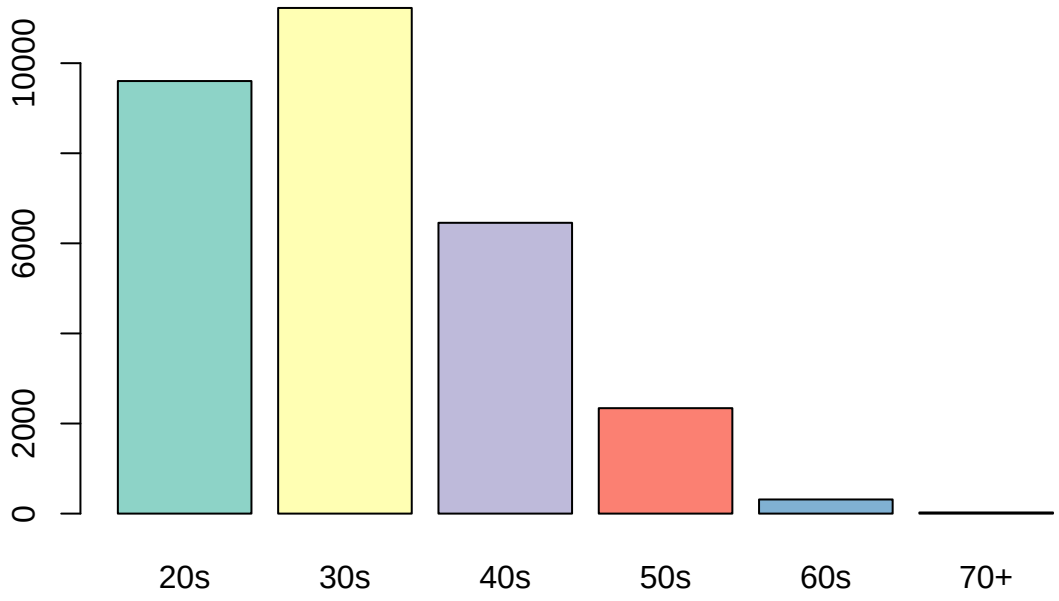
```
#Age groups divided into decades, then proportion table of each age group.
credit1$Age_Group <- cut(credit$Age,
                         breaks = c(20, 30, 40, 50, 60, 70, 80),
                         labels = c("20s", "30s", "40s", "50s", "60s", "70+"),
                         right = FALSE)

counts_Age <- table(credit1$Age_Group)
# Check distribution
prop.table(counts_Age)
```

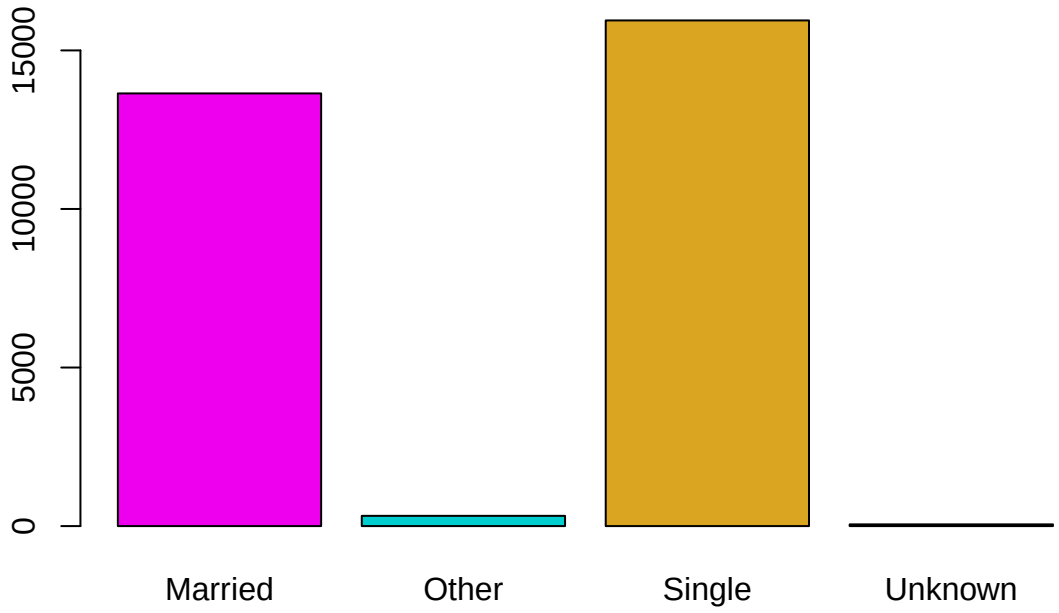
```
##
##          20s          30s          40s          50s          60s          70+
## 0.3204738862 0.3746370766 0.2154513599 0.0781244786 0.0104788920 0.0008343067
```

```
#Bar plot of age groups with custom colors
colors <- brewer.pal(6, "Set3")
```

```r
barplot(counts_Age, col = colors)
```




```r
#Counts for each marriage status.
counts_Marriage <- table(credit1$Marriage)
barplot(counts_Marriage, col = c("magenta2", "Cyan3", "goldenrod"))
```



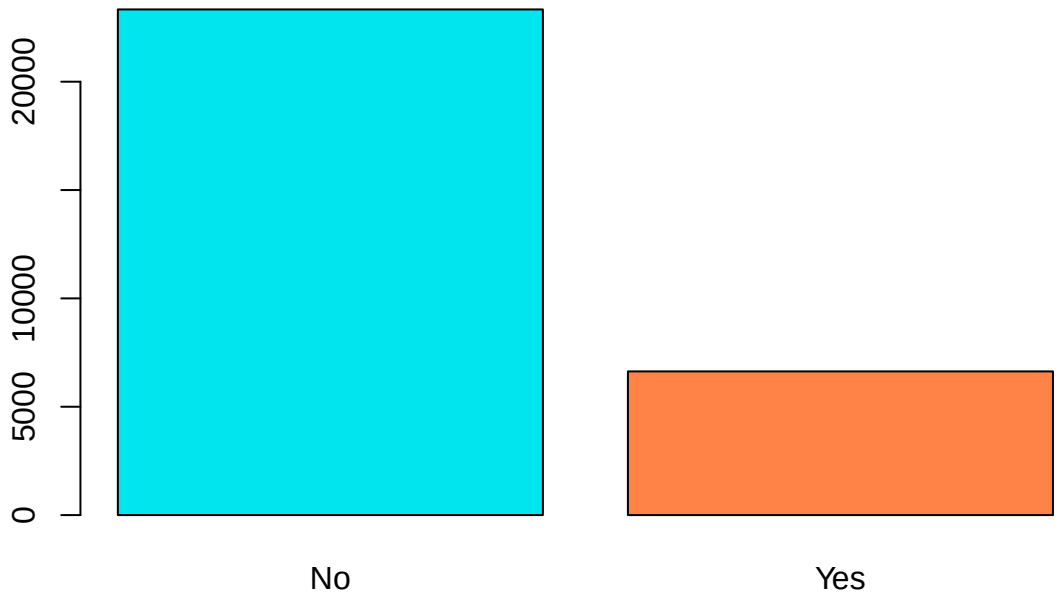

```
table(credit$Marriage)
```

```
##
##     0     1     2     3
##    54 13643 15945   323
```

```
#Proportion of each marriage status in table
prop.table(counts_Marriage)
```

```
##
##     Married       Other      Single     Unknown
## 0.455297847 0.010779242 0.532120808 0.001802102
```

```
counts_Default <- table(credit1$Default)
barplot(counts_Default, col = c("turquoise2", "sienna1"))
```




```
table(credit$Default)
```

```
##
##     0     1
## 23335  6630
```

```
prop.table(counts_Default)
```

```
##
##        No       Yes
## 0.7787419 0.2212581
```

```
table.default_gender <- table(credit1$Default, credit$Sex)
prop.table(table.default_gender, 2)
```

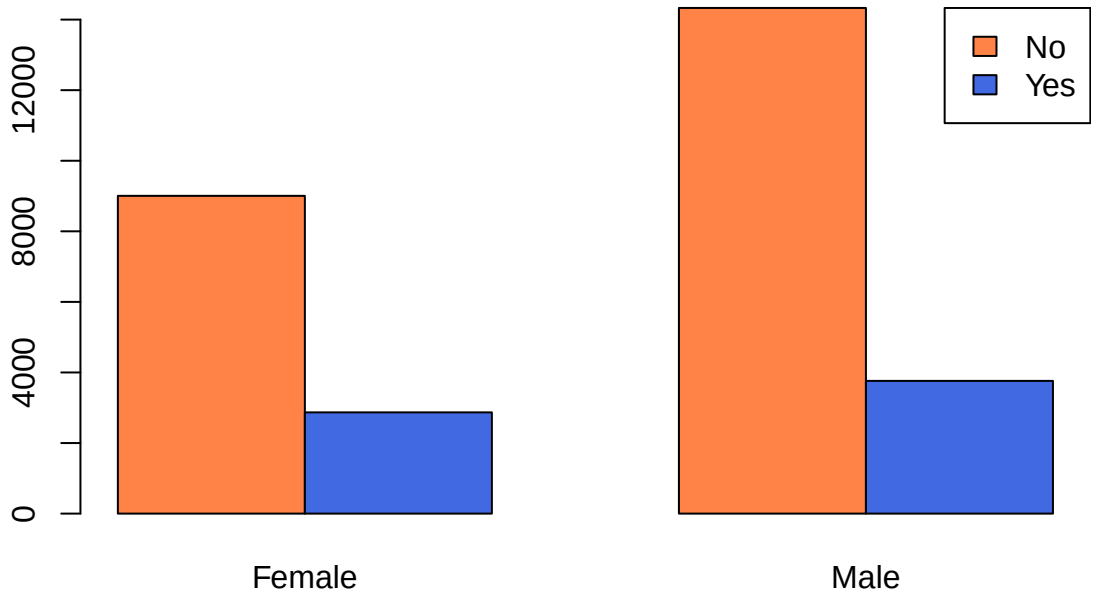
```
##
##               1         2
##   No  0.7583797 0.7921066
##   Yes 0.2416203 0.2078934
```

```
prop.table(table.default_gender, 1)
```

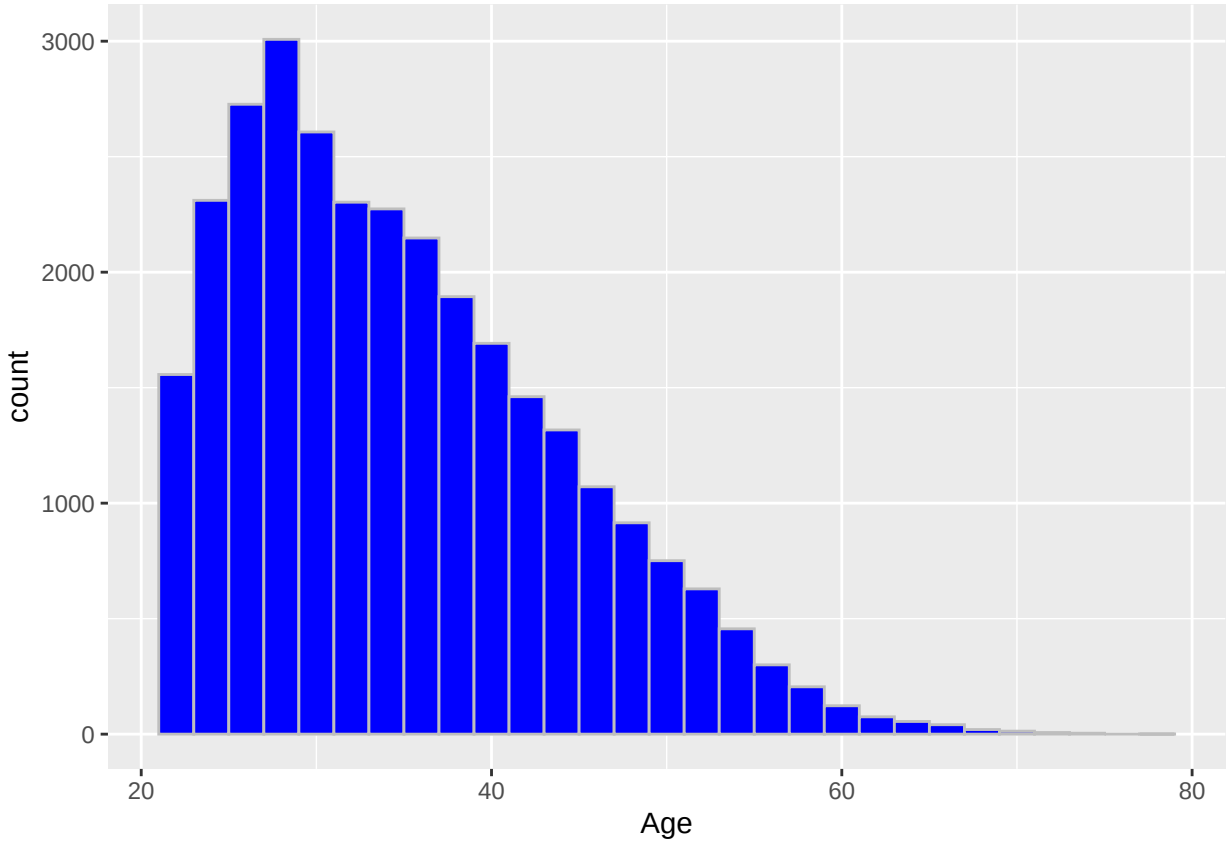
```
##
##              1        2
##   No  0.385901 0.614099
##   Yes 0.432730 0.567270
```

```
barplot(table.default_gender, col = c("sienna1", "royalblue"), beside = T,
        names.arg = c("Female", "Male"))
```

```
legend("topright", legend = c("No", "Yes"), fill = c("sienna1", "royalblue"))
```

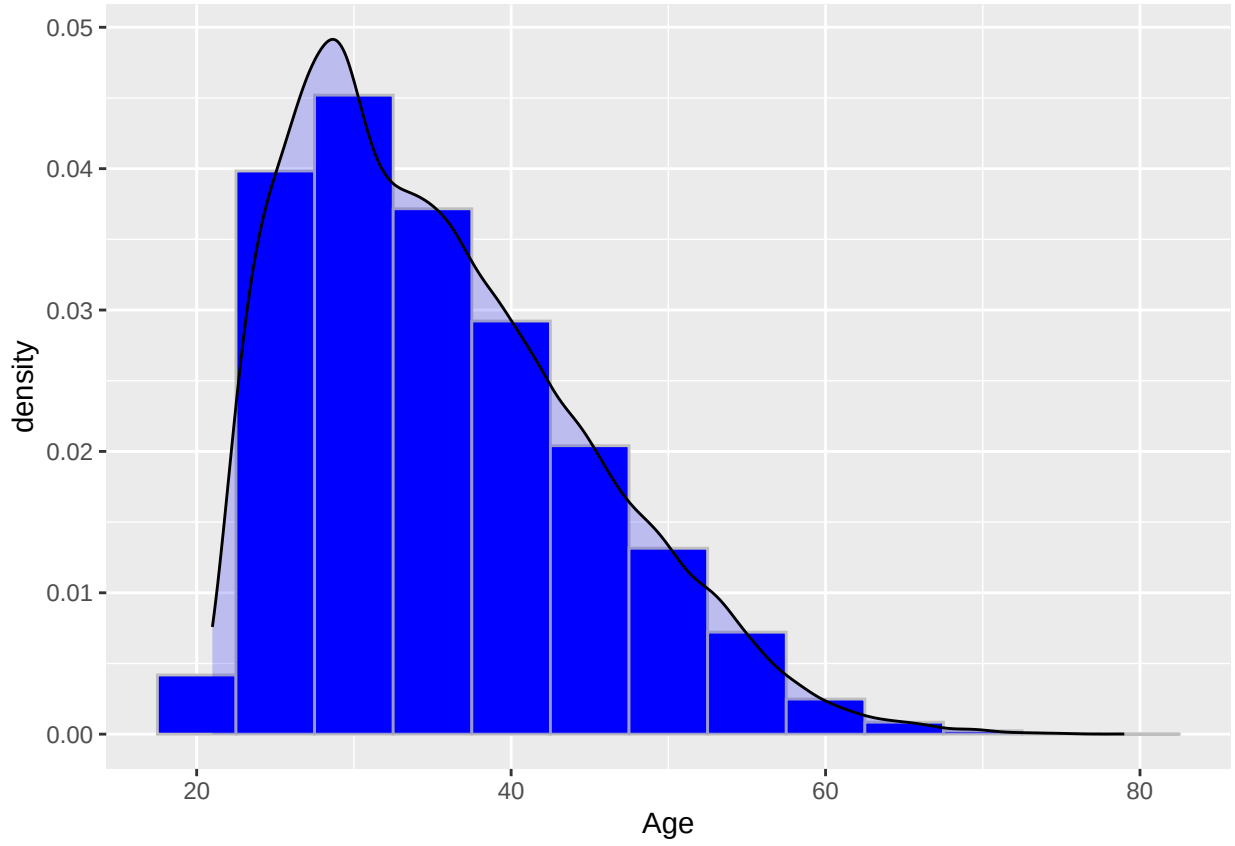



```
ggplot(data = credit, aes(x = Age)) + geom_histogram(fill = "Blue", col = "Grey", bins = 30)
```




```
ggplot(data = credit, aes(x = Age)) + geom_histogram(aes(y = ..density..), fill = "Blue", col = "Grey",
```

```
## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(density)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```
mean(credit1$Age)
```

```
## [1] 35.48797
```

Looking at the created charts and tables, the data has more females than males. In addition, the age group distribution is skewed to the right, meaning the data is represented by younger participants.

Scaling the Data

An important characterization for credit card payment is determining payment behavior. This can be done by creating a payment ratio, which would be the payment amount for one month divided by the bill amount for the preceding month. Before setting up the prediction model, a final data frame (credit_final) is created so payment ratios can be appropriately scaled and standardized.

```
credit_scaled <- credit %>%
  mutate(across(1:23, scale))
```

Train and Test Sets

Before creating prediction models, training and testing data sets are created. A training data set is a subset of examples used to train the model, while the testing data set is a subset used to test the training model.

```
#Initializes number generator.
set.seed(123)
#New sample created for the training and testing data sets. The data is split with 75% in training and
sample_split <- sample(c(TRUE, FALSE), nrow(credit_scaled), replace = TRUE, prob = c(0.75, 0.25))
train_set <- credit_scaled[sample_split, ]
test_set <- credit_scaled[!sample_split, ]
```

Sampling the Data

From the bar plots, it is clear there is an imbalance between those who default and those who did not in the data. This could cause issues in creating a prediction model, which would most likely skew towards predicting much more "No" answers since there are more within the sampled data. To solve this issue, oversampling and undersampling the training set data can be done. Oversampling duplicates random samples from the minority class, while undersampling randomly reduces samples from the majority class. Doing both helps to "even out" the bias and possibly improve the model's overall performance.

I chose to do only oversampling since the minority class (those who defaulted) is under 30%. The random oversampling is performed below:

```
credit_balance_train <- ovun.sample(Default ~., data = train_set, method = "over", seed = 123)$data

#Checks the table and proportion table of the resampled
table(credit_balance_train$Default)
```

```
##
##     0     1
## 17487 17211
```

```
prop.table(table(credit_balance_train$Default))
```

```
##
##         0         1
## 0.5039772 0.4960228
```

Now that the training and testing data sets are created and have been randomly sampled, prediction analysis methods such as logistic regression and random forest can be completed.

Logistic Regression

First, logistic regression is done to find the probability of default for an individual. Logistic regression models the probability that a response variable (Y) belongs to a particular category. This method uses maximum likelihood to fit the model in the range between 0 and 1.

Logistic regression is a classification method great for a yes/no response. A number closer to 1 represents "Yes", while a number closer to 0 represents "No".

A logistic regression model is created below, which is then used to predict the probabilities of credit card default for three individuals:

```
fit_glm2 <- glm(Default ~Limit_Bal+Sex+Education+Marriage+Age+Pay_Sep+Pay_Aug+
                  Pay_July+Bill_Amt_Sep+Bill_Amt_June+Pay_Amt_Sep+Pay_Amt_Aug+
                  Pay_Amt_July+Pay_Amt_June+Pay_Amt_May,
                  data = credit_balance_train, family = binomial())

summary(fit_glm2)
```

```
##
## Call:
## glm(formula = Default ~ Limit_Bal + Sex + Education + Marriage +
##     Age + Pay_Sep + Pay_Aug + Pay_July + Bill_Amt_Sep + Bill_Amt_June +
##     Pay_Amt_Sep + Pay_Amt_Aug + Pay_Amt_July + Pay_Amt_June +
##     Pay_Amt_May, family = binomial(), data = credit_balance_train)
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -0.20218    0.01207 -16.752  < 2e-16 ***
```

```
## Limit_Bal     -0.09364    0.01522  -6.151 7.70e-10 ***
## Sex           -0.03866    0.01169  -3.308 0.000941 ***
## Education     -0.06531    0.01263  -5.169 2.35e-07 ***
## Marriage      -0.09448    0.01292  -7.312 2.63e-13 ***
## Age            0.06966    0.01291   5.394 6.90e-08 ***
## Pay_Sep        0.57762    0.01507  38.326  < 2e-16 ***
## Pay_Aug        0.12628    0.01858   6.798 1.06e-11 ***
## Pay_July       0.07151    0.01698   4.211 2.54e-05 ***
## Bill_Amt_Sep  -0.23462    0.02799  -8.382  < 2e-16 ***
## Bill_Amt_June  0.17377    0.02920   5.950 2.68e-09 ***
## Pay_Amt_Sep   -0.17915    0.02201  -8.140 3.95e-16 ***
## Pay_Amt_Aug   -0.24859    0.02846  -8.735  < 2e-16 ***
## Pay_Amt_July  -0.05290    0.01602  -3.302 0.000958 ***
## Pay_Amt_June  -0.03749    0.01575  -2.381 0.017276 *
## Pay_Amt_May   -0.04752    0.01580  -3.007 0.002640 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 48099  on 34697  degrees of freedom
## Residual deviance: 42690  on 34682  degrees of freedom
## AIC: 42722
##
## Number of Fisher Scoring iterations: 5
```

```
stepAIC(fit_glm2)
```

```
## Start:  AIC=42722.02
## Default ~ Limit_Bal + Sex + Education + Marriage + Age + Pay_Sep +
##     Pay_Aug + Pay_July + Bill_Amt_Sep + Bill_Amt_June + Pay_Amt_Sep +
##     Pay_Amt_Aug + Pay_Amt_July + Pay_Amt_June + Pay_Amt_May
##
##                 Df Deviance   AIC
## <none>               42690 42722
## - Pay_Amt_June   1    42696 42726
## - Pay_Amt_May    1    42700 42730
## - Sex            1    42701 42731
## - Pay_Amt_July   1    42702 42732
## - Pay_July       1    42708 42738
## - Education      1    42717 42747
## - Age            1    42719 42749
## - Bill_Amt_June  1    42726 42756
## - Limit_Bal      1    42728 42758
## - Pay_Aug        1    42736 42766
## - Marriage       1    42744 42774
## - Bill_Amt_Sep   1    42763 42793
## - Pay_Amt_Sep    1    42775 42805
## - Pay_Amt_Aug    1    42792 42822
## - Pay_Sep        1    44234 44264
##
## Call:  glm(formula = Default ~ Limit_Bal + Sex + Education + Marriage +
##     Age + Pay_Sep + Pay_Aug + Pay_July + Bill_Amt_Sep + Bill_Amt_June +
##     Pay_Amt_Sep + Pay_Amt_Aug + Pay_Amt_July + Pay_Amt_June +
```

```
##     Pay_Amt_May, family = binomial(), data = credit_balance_train)
##
## Coefficients:
##   (Intercept)      Limit_Bal            Sex      Education       Marriage
##      -0.20218       -0.09364       -0.03866       -0.06531       -0.09448
##           Age        Pay_Sep        Pay_Aug       Pay_July   Bill_Amt_Sep
##       0.06966        0.57762        0.12628        0.07151       -0.23462
## Bill_Amt_June    Pay_Amt_Sep    Pay_Amt_Aug   Pay_Amt_July   Pay_Amt_June
##       0.17377       -0.17915       -0.24859       -0.05290       -0.03749
##   Pay_Amt_May
##      -0.04752
##
## Degrees of Freedom: 34697 Total (i.e. Null);  34682 Residual
## Null Deviance:       48100
## Residual Deviance: 42690     AIC: 42720
```

```
#VIF of the 2nd logistic regression model. VIF scores appear good.
vif(fit_glm2)
```

```
##     Limit_Bal           Sex     Education      Marriage           Age
##      1.541562      1.023403      1.130039      1.236390      1.286450
##       Pay_Sep       Pay_Aug      Pay_July  Bill_Amt_Sep Bill_Amt_June
##      1.558798      2.584196      2.245259      5.727019      6.327990
##   Pay_Amt_Sep   Pay_Amt_Aug  Pay_Amt_July  Pay_Amt_June   Pay_Amt_May
##      1.168651      1.155365      1.189827      1.084957      1.086287
```

In all, 15 variables were kept in the model due to statistical significance and VIF values under 7. VIF values under 7 indicate there is low-to-mid multicollinearity among the variables, though it can be argued that variables with a VIF of 5 or higher can be removed. Since the AIC increased when removign bill_amt_sep and bill_amt_june and made the model perform a little worse, I decided to leave in the two variables in the model.

```
pred_probs <- predict.glm(fit_glm2, newdata = test_set, type = "response")
#Displays the predictions for a few values.
head(pred_probs)
```

```
##         2         4         5         8        11        16
## 0.3924755 0.5176015 0.3568830 0.4143357 0.4638144 0.5648553
```

```
#Sorts predictions into their respective class (0 or 1) depending on their value.
pred <- ifelse(pred_probs<0.5, 0,1)
#Creates and displays the confusion matrix table based on the actual and predicted values.
confusion_table <- table(test_set$Default, pred)
confusion_table
```

```
##    pred
##        0    1
##   0 4310 1538
##   1  618 1022
```

```
#Creates the confusion matrix statistics for the logistic regression model.
cm_log <- confusionMatrix(confusion_table, positive = '1', mode = "everything")
#Saves the accuracy, precision, and recall values.
log_accuracy = accuracy(test_set$Default, pred)
log_precision = cm_log$byClass['Precision']
log_recall = cm_log$byClass['Recall']
log_pos_precision = cm_log$byClass['Neg Pred Value']
```

```r
#Prints the accuracy, precision, and recall values.
print(paste("Accuracy: ", round(log_accuracy,3)))
```

```
## [1] "Accuracy:  0.712"
```

```r
print(paste("Precision: ", round(log_precision,3)))
```

```
## [1] "Precision:  0.623"
```

```r
print(paste("Recall: ", round(log_recall,3)))
```

```
## [1] "Recall:  0.399"
```

```r
print(paste("Default Precision: ", round(log_pos_precision,3)))
```

```
## [1] "Default Precision:  0.737"
```

Once the model was run, the accuracy, precision, and recall were found for the prediction model. Accuracy describes how often the model is correct in its overall prediction. Precision identifies how often the model identifies those who default on their credit card out of all who do so, while recall identifies how often the model correctly identifies those who default on their credit card.Another way of describing precision and recall is precision is a measure of quality, while recall is a measure of quantity.

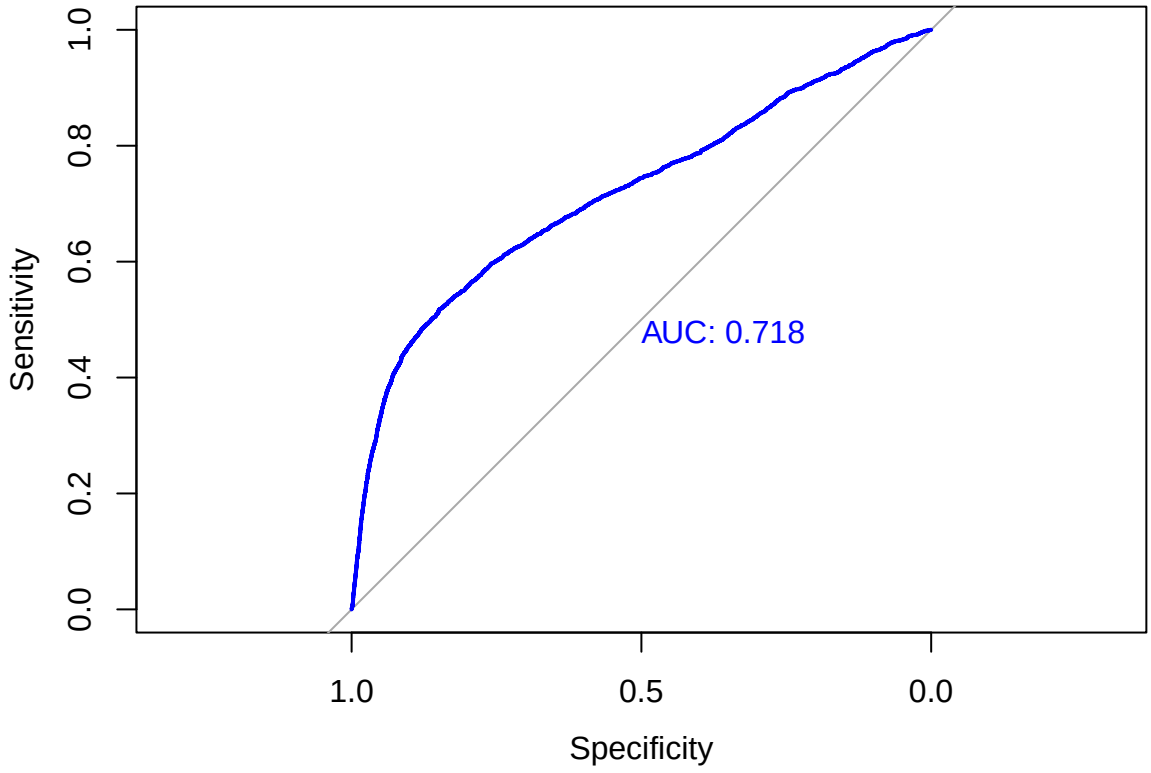
In the case of the logistic regression model, the accuracy was 71.2%, precision was 62.3%, and recall was 39.9%. The precision for predicting actual default cases correctly was 73.7%. Overall, the logistic regression model was fairly decent at its predicting whether a client would default.

The regression model's accuracy, specificity, and sensitivity can be improved by optimizing the cutoff point for the model. One way of doing so is using the Receiver Operating Characteristic (ROC) curve, which plots the true positive (sensitivity) against the false positive rate against various thresholds. The AUC curve can be used to measure the performance of the model, with a higher AUC number demonstrating better model performance. An AUC 0.8 and above indicates good model performance. The model has an AUC score of 0.718, which indicates acceptable model performance. This will be a factor to keep in mind for potential model improvements.

```r
prob <- predict(fit_glm2, type = "response")
# Create ROC curve
roc_obj <- roc(credit_balance_train$Default, prob, plot = TRUE, col = "blue", print.auc = TRUE)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```




```
# Find optimal cutoff (maximizes sensitivity + specificity)
#AUC performance = 0.718
opt <- coords(roc_obj, "best", ret = c("threshold", "sensitivity", "specificity"))
print(opt)
```

```
##   threshold sensitivity specificity
## 1 0.5479787   0.5172855    0.848802
```

```
#threshold = 0.548
#sensitivity = 0.517
#specificity = 0.849
```

The ROC curve supplies various cutoff values for the logistic regression model. From the ROC curve, the optimal cutoff point to maximize all three measurements (accuracy, specificity, and sensitivity) is 0.548, while the cutoffs to maximize sensitivity and specificity are 0.517 and 0.849, respectively. The optimal cutoff from the AUC curve, which maximizes both sensitivity and specificity, is 0.718. There is a tradeoff for each cutoff point, so it is up to the user to determine which is best for the purpose of the model. I chose to use 0.548 since I want to maximize both the overall accuracy of the model while accurately identifying those who will default on their credit card.

```
pred_new <- ifelse(pred_probs<0.548, 0,1)
#Creates and displays the confusion matrix table based on the actual and predicted values.
confusion_table_new <- table(test_set$Default, pred_new)
confusion_table_new
```

```
##    pred_new
##        0    1
##   0 5008  840
```

```
##   1  772  868
```

```
#Creates the confusion matrix statistics for the logistic regression model.
cm_log_opt <- confusionMatrix(confusion_table_new, positive = '1', mode = "everything")
#Saves the accuracy, precision, and recall values.
log_accuracy_opt = accuracy(test_set$Default, pred_new)
log_precision_opt = cm_log_opt$byClass['Precision']
log_recall_opt = cm_log_opt$byClass['Recall']
log_pos_precision_opt = cm_log_opt$byClass['Neg Pred Value']
#Prints the accuracy, precision, and recall values.
print(paste("Accuracy: ", round(log_accuracy_opt,3)))
```

```
## [1] "Accuracy:  0.785"
```

```
print(paste("Precision: ", round(log_precision_opt,3)))
```

```
## [1] "Precision:  0.529"
```

```
print(paste("Recall: ", round(log_recall_opt,3)))
```

```
## [1] "Recall:  0.508"
```

```
print(paste("Default Precision: ", round(log_pos_precision_opt,3)))
```

```
## [1] "Default Precision:  0.856"
```

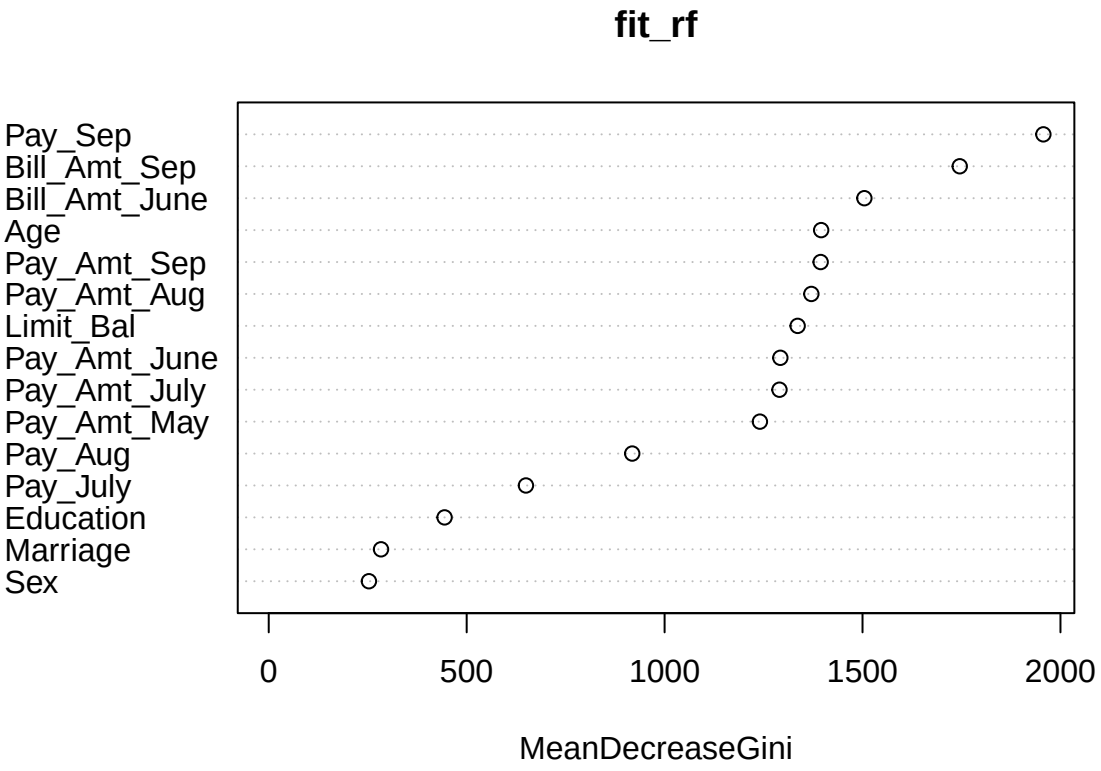
From the results, the accuracy, recall, and default precision all improved with the new threshold value, while precision had a decrease. Using this new threshold helps predict a default more accurately, but does so by incorrectly labeling someone as 'default' when they would not do so more often than the previous threshold mark. Overall, this model is kept since the overall accuracy improves and it's more important to identify those who will default.

Random Forest

Another prediction model used is random forest. Random forest is a classifying method consisting of many decision trees. By creating a "forest" of decision trees, the classifying model hopes to select it's best model by running many different decision trees and "takes the majority" to determine classification. To do so, random forest uses out-of-bag sampling.

A random forest model is created to determine the probability of credit card default:

```
set.seed(123)
#Random Forest for variables. mtry = 4 since there are 15 variables (square root of 10 is close to 4).
fit_rf <- randomForest(factor(Default) ~Limit_Bal+Sex+Education+Marriage+Age+Pay_Sep+Pay_Aug+
                  Pay_July+Bill_Amt_Sep+Bill_Amt_June+Pay_Amt_Sep+Pay_Amt_Aug+
                  Pay_Amt_July+Pay_Amt_June+Pay_Amt_May,
                  mtry = 4, data = credit_balance_train)
varImpPlot(fit_rf)
```




```r
#Predicts values in the test set.
predict_rf <- predict(fit_rf, test_set)
#Creates the confusion matrix table for the random forest model.
confusion_table_rf <- table(test_set$Default, predict_rf)
#Creates and displays the confusion matrix statistics for the random forest model.
cm_rf <- confusionMatrix(confusion_table_rf, positive = '1', mode = "everything")
cm_rf
```

```
## Confusion Matrix and Statistics
##
##    predict_rf
##        0    1
##   0 5387  461
##   1  919  721
##
##                Accuracy : 0.8157
##                  95% CI : (0.8067, 0.8244)
##     No Information Rate : 0.8421
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.4011
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.60998
##             Specificity : 0.85427
```

```
##          Pos Pred Value : 0.43963
##          Neg Pred Value : 0.92117
##               Precision : 0.43963
##                  Recall : 0.60998
##                      F1 : 0.51099
##              Prevalence : 0.15785
##          Detection Rate : 0.09629
##    Detection Prevalence : 0.21902
##       Balanced Accuracy : 0.73212
##
##        'Positive' Class : 1
##
```

```
#Saves the accuracy, precision, and recall values.
rf_accuracy = accuracy(test_set$Default, predict_rf)
rf_precision = cm_rf$byClass['Precision']
rf_recall = cm_rf$byClass['Recall']
rf_pos_precision = cm_rf$byClass['Pos Pred Value']
#Prints the accuracy, total precision, recall, and default precision values.
print(paste("Accuracy: ", round(rf_accuracy,3)))
```

```
## [1] "Accuracy:  0.816"
```

```
print(paste("Precision: ", round(rf_precision,3)))
```

```
## [1] "Precision:  0.44"
```

```
print(paste("Recall: ", round(rf_recall,3)))
```

```
## [1] "Recall:  0.61"
```

```
print(paste("Default Precision: ", round(rf_pos_precision,3)))
```

```
## [1] "Default Precision:  0.44"
```

From the input printed and the plot provided, it is seen that the pay amount and bill amount in September, as well as limit balance are important variables in determining credit card default. It can also be argued age, bill amount in August and bill amount in July are important variables in determining credit card default.

Looking at the confusion matrix, the random forest model's accuracy was 81.6%, precision was 44%, and recall was 61%. The precision for predicting actual default cases correctly was 44%. Overall, the random forest model was slightly better in its accuracy and recall. However, the logistic regression model performed better than the random forest model when predicting actual default cases.

Conclusion

In this project, logistic regression and random forest models were created to predict if an individual would default on their credit card.

First, the data was cleaned for accuracy and manipulated to view distributions and trends in the data. From the tables and plots created, the data had more females than males and was skewed in age, with participants below the age of 40 much more prevalent than participants over the age of 40.

Next, prediction models were created to predict an individual's chances of defaulting on their credit card. The first model used was a logistic regression model. This model was used to predict if an individual would default on their credit card based on their information. This model is great for predicting a Yes/No classification for individuals. From the model created, three individuals were created with their unique information. In the example above, all three individuals created had a good chance of not defaulting on their credit card.

A random forest model was also created to determine the most important variables in a prediction model, as well as to see the accuracy of the created model. From the results, the random forest model could accurately predict someone not defaulting on their credit card, but had a more difficult time accurately predicting when someone would default on their credit card.

When comparing the two models, the following table was created:

```
set.caption("Performance for Logistic Regression and Random Forest Models")
data.table = rbind(c(log_accuracy_opt, log_precision_opt, log_recall_opt, log_pos_precision_opt), c(rf_a
colnames(data.table) = c("Accuracy", "Precision", "Recall", "Default Precision")
rownames(data.table) = c("Logistic Regression", "Random Forest")

pander(data.table)
```

Table 1: Performance for Logistic Regression and Random Forest Models

	Accuracy	Precision	Recall	Default Precision
Logistic Regression	0.7847	0.5293	0.5082	0.8564
Random Forest	0.8157	0.4396	0.61	0.4396

Overall, it seems the logistic regression model has a higher precision and default precision rate than the random forest model, but does worse than the random forest model in accuracy and recall. This means the logistic regression model has less false positives than the random forest model, but also has more false negatives. Though the accuracy seems fairly high for the random forest prediction model, I am concerned with the false positive and false negative rates and the low default precision percentage.

Though these prediction models are acceptable, there is room for improvement, particularly in accurately predicting client that will default. I believe adding certain variables such as credit score, credit age, and credit card utilization can help improve the prediction models.

Thank you for viewing my project.

END